Software Design Specification (SDS)

AI-Based Layout Planning and Price Prediction System

Version 1.0 | March 2026

Project Name	AI-Based Layout Planning and Price Prediction System
Document Type	Software Design Specification (SDS)
Version	1.0
Date	March 2026
Frontend	PySide6 (Qt Desktop Application)
Backend	FastAPI (Python REST API)
ML Components	Layout Model + Price Prediction Model
Development Model	Agile (Iterative)

Table of Contents

1. Introduction

1.1 Purpose

The purpose of this document is to describe the software design of the AI-Based Layout Planning and Price Prediction System. This system allows users to input real estate plot parameters and receive an AI-generated architectural floor plan along with a predicted market price for the property.

The system processes the following inputs:

- Plot size (width × length in feet)
- Number of rooms and room types
- Plot facing direction (North/South/East/West)
- User preferences (Vastu, parking, number of floors)

The system produces the following outputs:

- AI-generated 2D floor plan (layout image)
- Predicted real estate price (based on ML model)
- Constraint validation feedback
- Downloadable PDF report

1.2 Document Conventions

- Headings are numbered for easy navigation.
- Diagrams are embedded directly in the document.
- Tables represent data structures and specifications.
- Technical terms are explained in the Glossary (Appendix B1).
- Dashed boxes in diagrams represent optional/background processes.

1.3 Intended Audience and Reading Suggestions

Audience	Purpose
Developers	Understand system modules, API contracts, and ML integration.
ML Engineers	Understand LayoutModel and PricePrediction model interfaces.
Project Managers	Review architecture, phase plan, and system scope.
Testers	Design test cases for API endpoints and ML model outputs.
Viva Examiners	Evaluate design quality, diagram accuracy, and completeness.

Readers should first read Section 1 (Introduction), then Section 2 (Analysis Model), then Section 3 (Design Model with all UML diagrams).

1.4 References

1. Software Engineering – Ian Sommerville
2. FastAPI Official Documentation – https://fastapi.tiangolo.com
3. PySide6 / Qt Documentation – https://doc.qt.io/qtforpython
4. Scikit-learn ML Library – https://scikit-learn.org

5. IEEE Software Documentation Standard
6. Real Estate Price Prediction using ML – Various Research Papers
7. Architectural Drawing Standards – Bureau of Indian Standards

2. Analysis Model

2.1 Methodology Used

The system follows a three-tier client-server architecture with an ML processing layer. The development approach is Agile and iterative, ensuring continuous testing and improvement at each stage.

Category	Technology / Tool
Frontend UI	PySide6 (Qt Widgets, Matplotlib)
Backend API	FastAPI (Python)
ML – Layout	LayoutModel (predictLayout, evaluateLayout)
ML – Price	PricePrediction Model (predictPrice, applyMultipliers)
Data Format	JSON (API communication)
Database	SQLite
Export	PDF (ReportLab / Matplotlib)
Version Control	Git / GitHub

The Agile model allows faster delivery of working modules and quick feedback integration from users and testers.

2.2 Use Case Diagram

The use case diagram below shows the interactions between the User and the system:

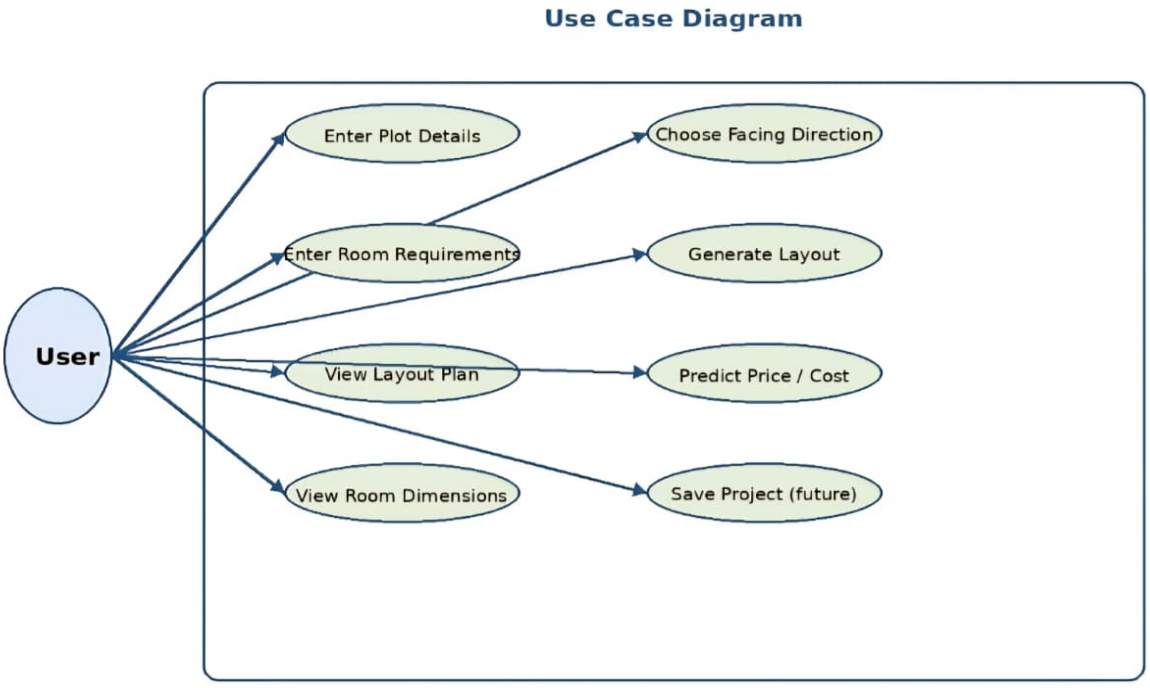

Figure 2.2 – Use Case Diagram (AI-Based Layout Planning and Price Prediction System)

Actors:

- End User (Homeowner / Property Buyer)
- FastAPI Backend System
- ML Model Engine

Main Use Cases:

- Enter Plot Size and Preferences
- Send API Request
- Generate Floor Plan Layout (ML)
- Predict Property Price (ML)
- Validate Layout Constraints
- Display Results on UI
- Download PDF Report

Use Case Specification

Field	Detail
Use Case Name	Generate Layout and Predict Price
Actor	End User
Precondition	User has entered plot size, room count, and facing direction.
Primary Flow	1. User inputs data via PySide6 UI. 2. Controller collects input and sends API request to FastAPI. 3. FastAPI triggers LayoutModel and PricePrediction Model. 4. ML models return layout data and predicted price. 5. FastAPI returns JSON results to UI. 6. PySide6 displays floor plan and price to user.
Post Condition	Floor plan and predicted price are displayed; user can download PDF.
Alternate Flow	If layout constraints fail, the system adjusts layout and retries.

2.3 ER Model

The ER diagram below shows the entity relationships in the AI Layout Price Prediction System database:

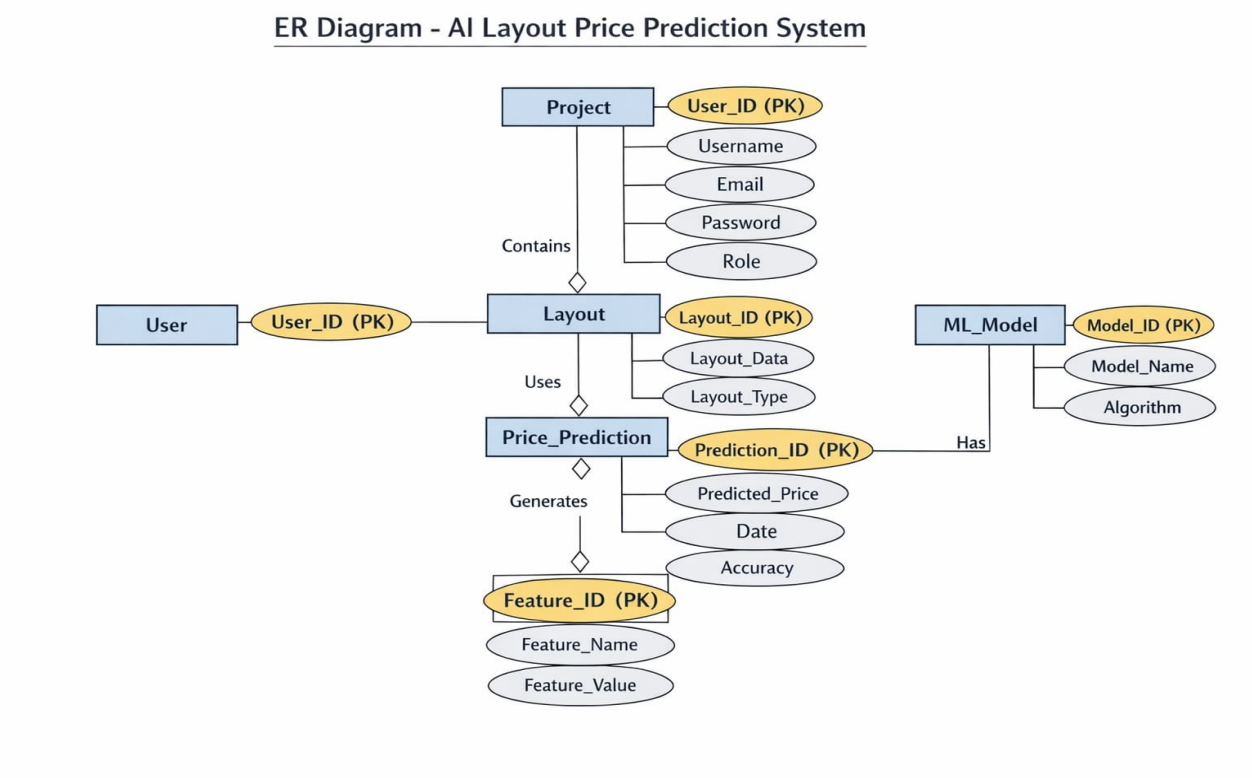

Figure 2.3 – ER Diagram: AI Layout Price Prediction System

Entities and Relationships:

Entity	Primary Key	Key Attributes
User	User_ID (PK)	Username, Email, Password, Role
Project	User_ID (PK)	Contains Layout (aggregation)
Layout	Layout_ID (PK)	Layout_Data, Layout_Type
Price_Prediction	Prediction_ID (PK)	Predicted_Price, Date, Accuracy
ML_Model	Model_ID (PK)	Model_Name, Algorithm
Feature	Feature_ID (PK)	Feature_Name, Feature_Value

Relationships:

- User → creates → Layout
- Layout → uses → Price_Prediction
- Price_Prediction → has → ML_Model
- Price_Prediction → generates → Feature
- Project → contains → Layout (aggregation)

2.4 Data Flow Diagram

DFD Level-0 (Context Diagram):

Input: Plot size, room preferences, facing direction

Process: AI Layout and Price Prediction System

Output: Floor plan image + Predicted price + Validation result

DFD Level-1 Processes:

Process	Input	Output
1. Input Collection	User enters data in PySide6 UI	Structured input object
2. API Request Handler	Input object from Controller	JSON request to FastAPI
3. Layout Generation (ML)	Plot data (size, rooms, direction)	Room coordinates / layout JSON
4. Price Prediction (ML)	Layout features + location data	Predicted price (INR)
5. Constraint Validation	Generated layout	Valid layout or adjustment request
6. Result Return	Layout JSON + price	JSON response to UI
7. Display & Export	Final layout + price	Rendered UI + downloadable PDF

2.5 Control Flow / Activity Diagram

The activity diagram (Figure 3.1) shows the flow of control from user input through ML processing to result display. The steps are described below:

Activity Flow Steps:

- Start
- User inputs plot size and preferences
- PySide6 Controller collects and validates input
- Send API Request to FastAPI Backend
- FastAPI triggers ML Layout Model → Generate Layout (plot analysis + optimal room placement)
- FastAPI triggers ML Price Predictor → Predict Price (material cost + price zone prediction)
- Validate Constraints? → If NO: Adjust Layout and re-validate
- If YES: Return Layout and Predicted Price to UI
- Display Result on PySide6 UI
- User downloads PDF report
- End

2.6 State Transition Diagram

State	Description
Idle	Application waiting for user input
Input Received	User has entered plot size, rooms, and facing direction
API Request Sent	Controller sends POST request to FastAPI backend
Layout Processing	LayoutModel computing optimal room placement
Price Predicting	PriceEstimator calculating property value
Validating	System checking layout constraints
Adjusting	Layout being corrected after failed constraint check
Results Ready	Layout and price returned to UI controller
Displayed	Floor plan and price shown to user in PySide6 window
Exported	PDF downloaded by user

3. Design Model

3.1 System Architecture & Activity Flow Diagram

The system follows a three-tier architecture: Frontend UI (PySide6) → Backend API (FastAPI) → ML Components (LayoutModel + PricePrediction). The diagram below also shows the complete activity flow from user input to result display.

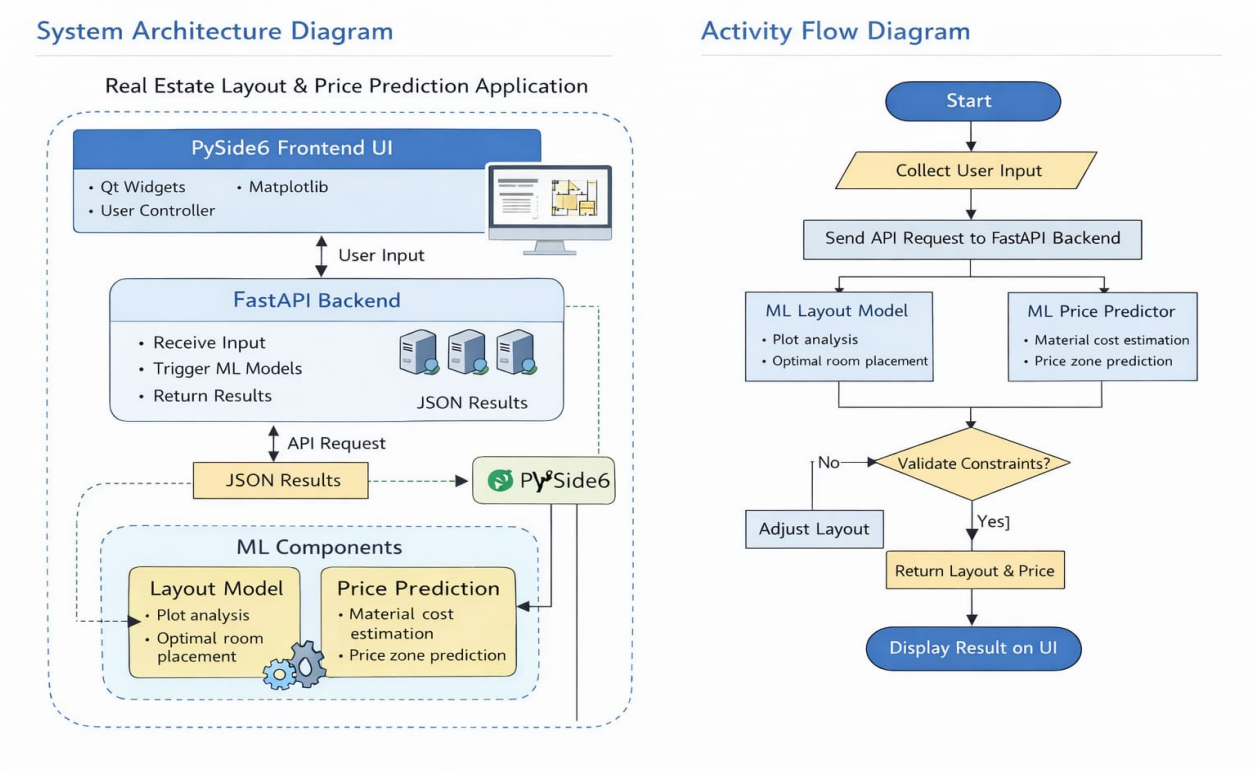

Figure 3.1 – System Architecture Diagram & Activity Flow Diagram (Real Estate Layout & Price Prediction Application)

3.1.1 Description of Architectural Layers:

PySide6 Frontend UI: Provides the desktop interface. Contains Qt Widgets for input, Matplotlib for rendering floor plans, and a User Controller for API communication.

FastAPI Backend: REST API server that receives input from the UI, triggers both ML models, and returns JSON results containing layout data and predicted price.

ML Components: Two independent ML models: (1) Layout Model for plot analysis and optimal room placement, and (2) Price Prediction Model for material cost estimation and price zone prediction.

Data Exchange: All communication between layers uses JSON format via HTTP POST requests.

3.2 UML Class Diagram

The UML Class Diagram defines all system classes, their attributes, methods, and inter-class relationships:

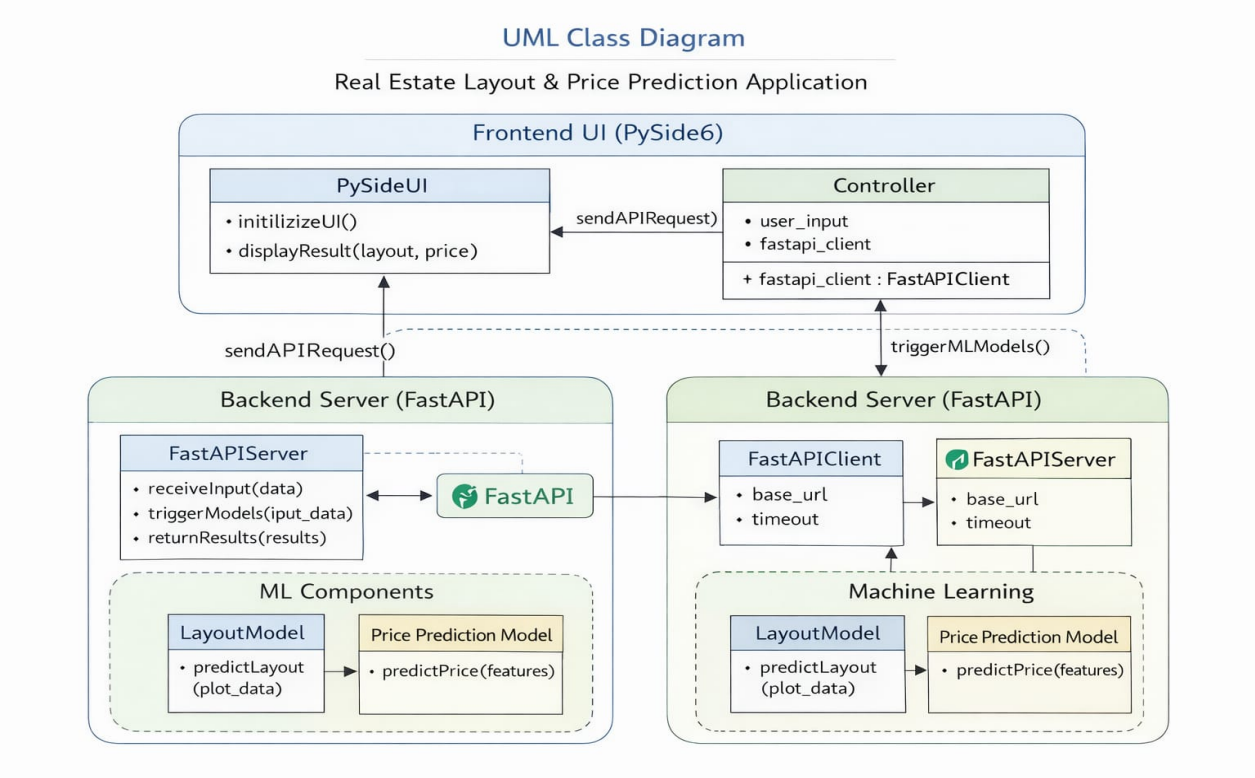

Figure 3.2 – UML Class Diagram (Real Estate Layout & Price Prediction Application)

Class Descriptions:

Class	Layer	Key Attributes / Methods
PySideUI	Frontend	initializeUI(), displayResult(layout, price)
Controller	Frontend	user_input, fastapi_client : FastAPIClient + fastapi_client : FastAPIClient
FastAPIServer	Backend	receiveInput(data), triggerModels(input_data), returnResults(results)
FastAPIClient	Backend	base_url, timeout
LayoutModel	ML	predictLayout(plot_data)
PricePredictionModel	ML	predictPrice(features)
UserInterface	Frontend	start(), showResult()
InputHandler	Frontend	getPlotData(), getUserPreferences()
ResultDisplay	Frontend	displayLayout(), showPrice()
LayoutGenerator	Backend/ML	generateLayout(), placeDoors()
MLPredictor	ML	predictZones(), evaluateLayout()
PriceEstimator	ML	calculateCost(), applyMultipliers()

Key Relationships:

- Controller → sends request to → FastAPIServer (dependency)
- FastAPIServer → triggers → LayoutModel and PricePredictionModel (association)
- FastAPIClient → communicates with → FastAPIServer (association)
- UserInterface → inherits from → InputHandler and ResultDisplay (generalization)
- LayoutGenerator → uses → MLPredictor and PriceEstimator (association)

3.3 Sequence Diagram

The Sequence Diagram shows the time-ordered interaction between system components during a floor plan generation request:

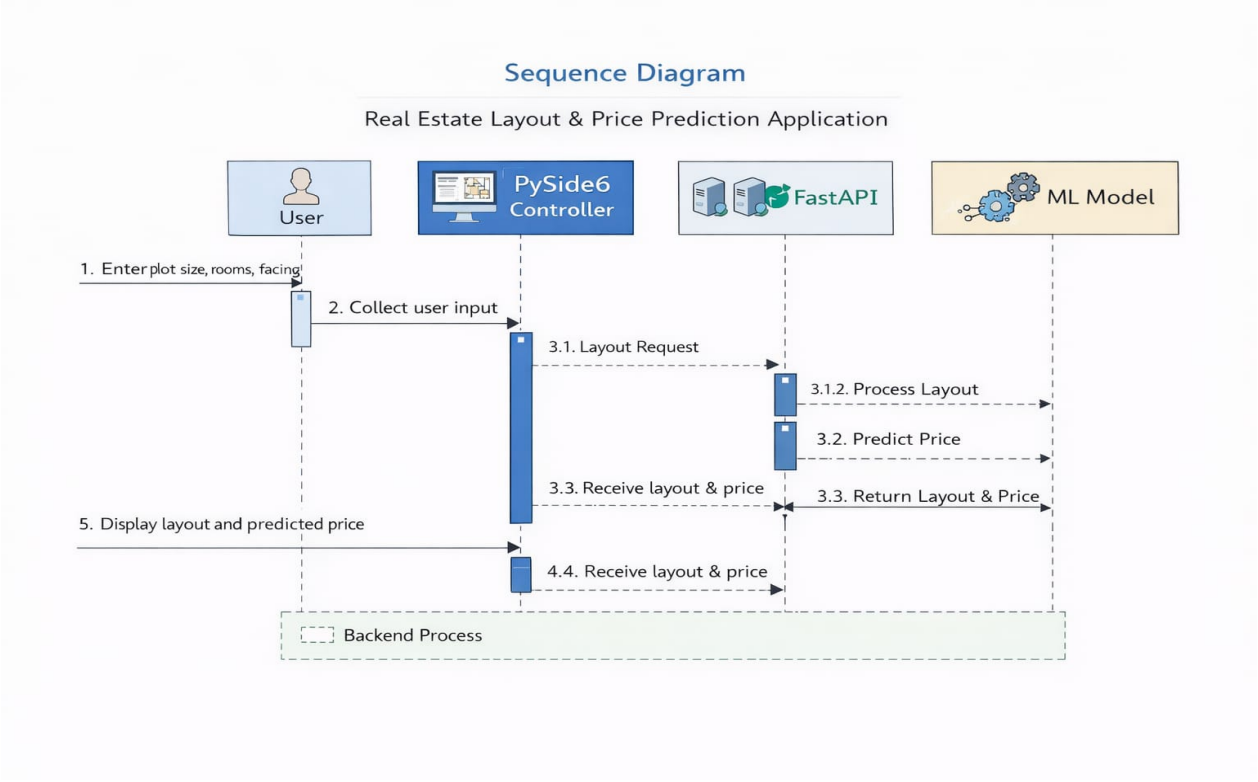

Figure 3.3 – Sequence Diagram (Real Estate Layout & Price Prediction Application)

Sequence Steps:

Step	From	To	Message
1	User	PySide6 Controller	Enter plot size, rooms, facing direction
2	PySide6 Controller	PySide6 Controller	Collect and validate user input
3.1	PySide6 Controller	FastAPI Backend	Layout Request (HTTP POST with JSON data)
3.1.2	FastAPI Backend	ML Model	Process Layout (trigger LayoutModel)
3.2	FastAPI Backend	ML Model	Predict Price (trigger PricePrediction Model)
3.3	ML Model	FastAPI Backend	Return Layout & Price (JSON response)
3.3	FastAPI Backend	PySide6 Controller	Receive layout & price
4.4	PySide6 Controller	PySide6 Controller	Receive layout & price (internal)
5	PySide6 Controller	User	Display layout and predicted price on UI

3.4 Database Design

3.4.1 Data Dictionary

Field Name	Table	Type	Description
User_ID	Users	Integer (PK)	Unique user identifier
Username	Users	String	Login username

Email	Users	String	User email address
Password	Users	String (hashed)	Encrypted password
Role	Users	String	User role (admin/user)
Layout_ID	Layouts	Integer (PK)	Unique layout identifier
Layout_Data	Layouts	JSON/Blob	Room coordinates and dimensions
Layout_Type	Layouts	String	Type: residential/commercial
Prediction_ID	Predictions	Integer (PK)	Unique prediction ID
Predicted_Price	Predictions	Float	Estimated price in INR
Accuracy	Predictions	Float	Model accuracy percentage
Model_ID	ML_Models	Integer (PK)	ML model identifier
Model_Name	ML_Models	String	e.g. RandomForest_v2
Algorithm	ML_Models	String	e.g. Random Forest Regression
Feature_ID	Features	Integer (PK)	Feature record identifier
Feature_Name	Features	String	e.g. plot_area, num_rooms
Feature_Value	Features	Float	Numeric feature value

3.4.2 Normalization

Normal Form	Action Applied
1NF	All fields contain atomic values. No repeating groups in any table.
2NF	Separate tables: Users, Layouts, Predictions, ML_Models, Features. No partial dependencies.
3NF	Transitive dependencies removed. Model accuracy stored in ML_Models, not Predictions.

Final Database Tables: Users | Layouts | Price_Predictions | ML_Models | Features

3.5 Component Design

3.5.1 Module Architecture (Class Hierarchy)

Module Responsibilities:

Module	Responsibility
UserInterface	Base class providing start() and showResult() interface
InputHandler	Collects plot data and user preferences from UI widgets
ResultDisplay	Renders layout image and displays price in UI
LayoutGenerator	Calls MLPredictor to generate room layout, places doors
MLPredictor	Predicts room zones and evaluates generated layout
PriceEstimator	Calculates construction cost and applies location multipliers
FastAPIServer	Receives HTTP requests, routes to models, returns JSON
FastAPIClient	Sends HTTP requests from Controller to backend
Controller	Orchestrates UI input → API request → result display

3.6 Interface Design

3.6.1 Desktop Application Interface (PySide6)

The PySide6 desktop interface contains the following components:

UI Component	Description
Plot Size Input	Width and Length fields (in feet)
Room Count Input	Number of BHK rooms (1BHK to 5BHK)
Facing Direction	Dropdown: North / South / East / West
Generate Button	Triggers Controller → API request → ML processing
Layout Display Panel	Matplotlib canvas showing generated 2D floor plan
Price Display Label	Shows predicted price in INR with breakdown
Floor Tabs	Separate tabs: Ground Floor | First Floor | Cost Summary
Validate Feedback	Shows constraint warnings or success message
Download PDF Button	Exports floor plan + price report as PDF

3.6.2 FastAPI Backend Endpoints

Endpoint	Method	Input	Output
/generate-layout	POST	plot_data JSON	layout JSON + room coordinates
/predict-price	POST	feature JSON	predicted_price (INR)
/validate-layout	POST	layout JSON	valid: bool, adjustments: list
/health	GET	None	API status message

Appendix B1 – Glossary

Term	Definition
AI	Artificial Intelligence – systems that simulate human intelligence
ML	Machine Learning – AI technique for learning patterns from data
FastAPI	Modern Python web framework for building REST APIs
PySide6	Python binding for Qt6, used for desktop GUI development
UML	Unified Modeling Language – standardized software design notation
ER Diagram	Entity-Relationship Diagram – database structure visualization
JSON	JavaScript Object Notation – lightweight data interchange format
API	Application Programming Interface – service communication contract
BHK	Bedroom, Hall, Kitchen – Indian residential unit classification
Vastu	Vastu Shastra – Indian directional architectural science
Layout Model	ML model that generates optimal room placement from plot data
Price Prediction	ML model that estimates property market value
Constraint	Rule that a generated layout must satisfy (e.g., minimum room size)
Matplotlib	Python plotting library used for rendering 2D floor plans
SDS	Software Design Specification – this document
PK	Primary Key – unique identifier for a database record
REST	Representational State Transfer – web API architectural style

Appendix B2 – To Be Determined (TBD) List

#	Item	Status
1	Final ML algorithm selection (Random Forest vs XGBoost for price prediction)	TBD
2	Real estate price dataset source and preprocessing pipeline	TBD
3	Cloud deployment configuration (AWS / Azure / Render)	TBD
4	User authentication and login system for web version	TBD
5	3D visualization of generated floor plans	TBD
6	Mobile application version (Android / iOS)	TBD
7	Integration with CAD export formats (DXF / DWG)	TBD
8	Multi-city price prediction support with location zones	TBD
9	Real-time collaborative editing of floor plans	TBD